RFM ANALYSIS: TARGET CUSTOMER SEGMENTATION

1. Introduction to the situation

CosmeticWorld is a global retail chain for cosmetics and body care products. After 4 years of operation (2020 - 2023), the system has recorded over 50,000 transactions.

Currently, CosmeticWorld is planning for the "New Year Rejuvenation 2024" campaign. However, if this campaign is run, promotional emails cannot be sent indiscriminately because the messaging cost would be very high and it would bother customers who are not interested in receiving campaign information. Therefore, it is necessary to classify customers into different segments based on their shopping behavior to optimize the outreach strategy.

2. Problem statement

- Which customer group brings the highest profit to CosmeticWorld?
- Who are these customers? Where are they? And what do they buy?

3. Data used

- EcomSales.csv: Sales information.
- Customer.csv: Customer information.
- Product.csv: Product information
- Region.csv: Geographical location information.

I. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from datetime import datetime

pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.expand_frame_repr', None)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

II. Data Preprocessing

```
# Load data

df_sales = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/EcomSales.csv')
df_customers = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Customer.csv')
df_products = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Product.csv')
df_regions = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Region.csv')
```

2.1. Inspect Data

```
# Check data

def check_df(df):
  print('=' * 50)
  print('DATAFRAME INFORMATION')
  print('=' * 50)
  df.info()

  print('=' * 50)
  print('CHECK FOR NULL VALUES')
  print('=' * 50)
  print(df.isna().sum())

  print('=' * 50)
  print('CHECK FOR DUPLICATE DATA')
  print('=' * 50)
  print(f" Number of duplicates: {df.duplicated().sum()}")

  print('=' * 50)
  print('DESCRIPTIVE STATISTICS')
  print('=' * 50)
  print(df.describe().T)
```

2.1.1. Inspect df_sales

```
check_df(df_sales)
```

```
sns.boxplot(data = df_sales, x = 'Quantity')
```

```
sns.boxplot(data = df_sales, x = 'Sales')
```

```
sns.boxplot(data = df_sales, x = 'Profit')
```

Note: Columns `Sales`, `Quantity`, `Profit` have outliers

2.1.2. Inspect df_customers

```
check_df(df_customers)
```

```
sns.boxplot(data = df_customers, x = 'AnnualIncome')
```

Note: `Gender` column has nulls and `AnnualIncome` has outliers

Orders with sales identified as outliers are likely to be from potential customers, a small number who need more special care than the majority.

And large revenue also means a large quantity of products in one order (in the `Quantity` column) and `Profit` also fluctuates strongly, leading to outlier results.

Outliers in the `AnnualIncome` column of df_customers could be potential customers.

2.1.3. Inspect df_products

```
check_df(df_products)
```

2.1.4. Inspect df_regions

```
check_df(df_regions)
```

2.2. Data Preprocessing & Cleaning

- Change data types
- Fix typo errors
- Handle duplicate data
- Handle null values
- Handle outliers
- ...

2.3.1. Handle Null Values

```
df_customers['Gender'].fillna('Undefined', inplace = True)
df_customers[df_customers['Gender'] == 'Undefined'].head()
```

2.3.2. Handle data entry errors

Unusual codes in df_sales

```
import re

# Format the ProductCode column in df_products (1 uppercase letter at the beginning followed by 6 digits, e.g., P000001...)
pattern = r'^[A-Z]\d{6}$'

# Find data that does not conform to the above format
non_conforming_product_codes = df_sales[~df_sales['ProductCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_product_codes)
```

```
# Format the RegionCode column in df_region (1 uppercase letter at the beginning followed by 4 digits, e.g., R0001...)
pattern = r'^[A-Z]\d{4}$'

# Find data that does not conform to the above format
non_conforming_region_codes = df_sales[~df_sales['RegionCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_region_codes)
```

For data with codes that differ from the dimension table format, it is necessary to check with upstream sources or relevant parties before modifying these unusual product and region codes.

2.3. Connect dataframes

```
# 1.1 Merge data from different tables
df_merged = pd.merge(df_customers, df_sales, on='CustomerID', how='left')
df_merged = pd.merge(df_merged, df_products, on='ProductCode', how='left')
df = pd.merge(df_merged, df_regions, on='RegionCode', how='left')

# 1.2 Convert date format
df['BirthDate'] = pd.to_datetime(df['BirthDate'])
df['OrderDate'] = pd.to_datetime(df['OrderDate'])

# 1.3 Calculate actual revenue (after discount)
df['Revenue'] = df['Sales'] * (1 - df['Discount'])

display(df.head())
```

```
df_rfm = df[['CustomerID', 'OrderDate','OrderID','Revenue']]
df_rfm.head(15)
```

III. RFM Analysis

3.1. Grouping and checking quintile intervals

```
df_rfm_groupby = df_rfm.groupby(by = 'CustomerID', as_index=False).agg(max_date = ('OrderDate', 'max') ,
                                                       frequency = ('OrderID', 'nunique') ,
                                                       monetary = ('Revenue', 'sum')
                                                       )
df_rfm_groupby['recency'] = (datetime.now() - df_rfm_groupby['max_date']).dt.days
df_rfm_groupby.head()
```

```
df_rfm_groupby['R_score'] = pd.qcut(df_rfm_groupby['recency'] , q = 5, duplicates = 'drop')
df_rfm_groupby['F_score'] = pd.qcut(df_rfm_groupby['frequency'] , q = 5, duplicates = 'drop')
df_rfm_groupby['M_score'] = pd.qcut(df_rfm_groupby['monetary'] , q = 5, duplicates = 'drop')
```

```
df_rfm_groupby['R_score'].unique()
```

```
df_rfm_groupby['F_score'].unique() # data is skewed, so there are only 2 intervals, need to handle separately
```

```
df_rfm_groupby['M_score'].unique()
```

3.2. Handle quintile intervals for Frequency field

```
df_frequency = df_rfm_groupby[['frequency']].drop_duplicates().sort_values('frequency')
df_frequency['F_score'] = pd.qcut(df_frequency['frequency'], q=5, duplicates='drop')
df_frequency
```

```
df_frequency['F_score'].unique() # Now has 5 quintile intervals
```

3.3. Assign R_score, F_score, M_score, RFM_score values

```
df_rfm_groupby['R_score'] = pd.qcut(df_rfm_groupby['recency'] , q = 5, labels = [5, 4, 3, 2, 1], duplicates = 'drop')
df_rfm_groupby['M_score'] = pd.qcut(df_rfm_groupby['monetary'] , q = 5, labels = [1, 2, 3, 4, 5], duplicates = 'drop')
```

```
df_frequency['F_score'] = pd.qcut(df_frequency['frequency'], q=5, labels = [1, 2, 3, 4, 5], duplicates='drop')
df_rfm_groupby = df_rfm_groupby.merge(df_frequency, on = 'frequency', how = 'left')
df_rfm_groupby
```

```
df_rfm_groupby.columns
```

```
df_rfm_groupby = df_rfm_groupby[['CustomerID', 'max_date', 'frequency', 'monetary', 'recency', 'R_score', 'M_score', 'F_score_y']]
df_rfm_groupby
```

```
df_rfm_groupby.rename(columns = {'F_score_y': 'F_score'}, inplace = True)
df_rfm_groupby
```

```
df_rfm_groupby['R_score'] = df_rfm_groupby['R_score'].astype(int)
df_rfm_groupby['F_score'] = df_rfm_groupby['F_score'].astype(int)
df_rfm_groupby['M_score'] = df_rfm_groupby['M_score'].astype(int)
df_rfm_groupby['RFM_Score'] = df_rfm_groupby['R_score'] * 100 + df_rfm_groupby['F_score'] * 10 + df_rfm_groupby['M_score']
df_rfm_groupby
```

3.4. Customer Segmentation

```
df_rfm_groupby['RFM_Segment'] = 'Other'  # Default label for rows not matching any segment

# Champions
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([555, 554, 544, 545, 454, 455, 445]), 'RFM_Segment'] = 'Champions'
```

```
# Loyal
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([543, 444, 435, 355, 354, 345, 344, 335]), 'RFM_Segment'] = 'Loyal'

# Potential Loyalist
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([553, 551, 552, 541, 542, 533, 532, 531, 452, 451
                                                     , 442, 441, 431, 453, 433, 432, 423, 353, 352
                                                     , 351, 342, 341, 333, 323]), 'RFM_Segment'] = 'Potential Loyalist'

# Promising
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([525, 524, 523, 522, 521, 515, 514, 513
                                                     , 425, 424, 413, 414, 415, 315, 314, 313]), 'RFM_Segment'] = 'Promising'

# New Customers
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([512, 511, 422, 421, 412, 411, 311]), 'RFM_Segment'] = 'New Customers'

# Need Attention
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([535, 534, 443, 434, 343, 334, 325, 324]), 'RFM_Segment'] = 'Need Attention'

# About To Sleep
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([331, 321, 312, 221, 213, 231, 241, 251]), 'RFM_Segment'] = 'About To Sleep'

# At Risk
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([255, 254, 245, 244, 253, 252, 243, 242, 235
                                                     , 234, 225, 224, 153, 152, 145, 143, 142
                                                     , 135, 134, 133, 125, 124]), 'RFM_Segment'] = 'At Risk'

# Cannot Lose Them
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([155, 154, 144, 214, 215, 115, 114, 113]), 'RFM_Segment'] = 'Cannot Lose Them'

# Hibernating customers
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([332, 322, 233, 232, 223, 222, 132, 123, 122, 212, 211]), 'RFM_Segment'] = 'Hibernating customers'

# Lost customers
df_rfm_groupby.loc[df_rfm_groupby['RFM_Score'].isin([111, 112, 121, 131, 141, 151]), 'RFM_Segment'] = 'Lost customers'

# Display the DataFrame with the new RFM_Segment column
df_rfm_groupby
```

VISUALIZATION

```
df_profit = df_sales.groupby(by = 'CustomerID', as_index=False).agg(total_profit = ('Profit', 'sum'))
df_profit
```

```
df_rfm_groupby = df_rfm_groupby.merge(df_profit, on = 'CustomerID', how = 'left')
df_rfm_groupby
```

```
treemap_input = df_rfm_groupby.groupby('RFM_Segment', as_index=False).agg(
    count_cus=('CustomerID', 'nunique'),
    total_rev=('monetary', 'sum'),
    total_profit=('total_profit', 'sum')
)

# Customer quantities Treemap
fig_count = px.treemap(treemap_input, path=['RFM_Segment'], values='count_cus',
                       title='Customer Allocation by Group')
fig_count.show()

# Revenue contribution Treemap
fig_rev = px.treemap(treemap_input, path=['RFM_Segment'], values='total_rev',
                     title='Revenue contribution by customer group')

fig_rev.show()

# Profit contribution Treemap

fig_profit = px.treemap(treemap_input, path=['RFM_Segment'], values='total_profit',
                        title='Profit contribution by customer group')
fig_profit.show()
```

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ======================================================
# Prepare Data
# ======================================================
pie_input = (
    df_rfm_groupby
    .groupby('RFM_Segment', as_index=False)
    .agg(
        count_cus=('CustomerID', 'nunique'),
        total_profit=('total_profit', 'sum')
    )
)

# Descending
count_df = (
    pie_input
    .sort_values('count_cus', ascending=False)
    .reset_index(drop=True)
)

profit_df = (
    pie_input
```

```
    .sort_values('total_profit', ascending=False)
    .reset_index(drop=True)
)

# Rank
count_df['label'] = [
    f"{i+1}. {seg}"
    for i, seg in enumerate(count_df['RFM_Segment'])
]

profit_df['label'] = [
    f"{i+1}. {seg}"
    for i, seg in enumerate(profit_df['RFM_Segment'])
]

# ====================================================
# Create Figure
# ====================================================
fig = make_subplots(
    rows=1,
    cols=2,
    specs=[[{'type': 'domain'}, {'type': 'domain'}]],
    horizontal_spacing=0.12,

    subplot_titles=(
        "<b>Customer quantities</b>",
        "<b>Profit</b>"
    )
)

# ====================================================
# Donut 1 - Customer Count
# ====================================================
fig.add_trace(
    go.Pie(
        labels=count_df['label'],
        values=count_df['count_cus'],

        sort=False,
        direction='clockwise',
        rotation=90,

        hole=0.42,

        textinfo='percent',
        textposition='inside',

        insidetextorientation='radial',
```

```
        hovertemplate=
        "<b>%{label}</b><br>" +
        "Customer quantities: %{value:,.0f}<br>" +
        "Percentage: %{percent}<extra></extra>"
    ),
    row=1,
    col=1
)

# ====================================================
# Donut 2 - Profit
# ====================================================
fig.add_trace(
    go.Pie(
        labels=profit_df['label'],
        values=profit_df['total_profit'],

        sort=False,
        direction='clockwise',
        rotation=90,

        hole=0.42,

        textinfo='percent',
        textposition='inside',

        insidetextorientation='radial',

        hovertemplate=
        "<b>%{label}</b><br>" +
        "Profit: %{value:,.0f}<br>" +
        "Percentage: %{percent}<extra></extra>"
    ),
    row=1,
    col=2
)

# ====================================================
# Layout
# ====================================================
fig.update_layout(

    width=1000,
    height=500,

    showlegend=True,
```

```
        legend=dict(
            orientation='v',
            y=0.5,
            yanchor='middle',
            x=1.02,
            xanchor='left',
            font=dict(size=12)
        ),

        margin=dict(
            t=50,
            l=30,
            r=280,
            b=120
        ),

        uniformtext_minsize=10,
        uniformtext_mode='hide'
    )

    # ====================================================
    # Move subplot titles to bottom
    # ====================================================
    for ann in fig.layout.annotations:
        ann.update(
            y=-0.08,
            font=dict(
                size=22,
                color='#1f3b64'
            )
        )

    fig.show()
```

3.5. Analyze information on the most profitable customer segment, their orders, and their geographical location

```
    df_rfm_groupby['RFM_Segment'] = df_rfm_groupby['RFM_Segment'].astype(str)
    # Identify the most profitable segment
    most_profitable_segment_name = treemap_input.loc[treemap_input['total_profit'].idxmax(), 'RFM_Segment']

    print(f"The most profitable customer segment is: {most_profitable_segment_name}")

    # Get CustomerIDs for the most profitable segment
    most_profitable_customer_ids = df_rfm_groupby[df_rfm_groupby['RFM_Segment'] == most_profitable_segment_name]['CustomerID'].unique()
```

```
# Filter the main dataframe (df) for these customers
df_profitable_customers = df[df['CustomerID'].isin(most_profitable_customer_ids)]

# 1. Who are these customers?
print(f"\n1. Customers in the '{most_profitable_segment_name}' segment:")
print(f"Number of unique customers: {len(most_profitable_customer_ids)}")
print("Top 5 customers (example):")
print(df_profitable_customers[['CustomerID', 'FirstName', 'LastName', 'EmailAddress']].drop_duplicates().head())

# 2. Where are they?
print(f"\n2. Location of customers in the '{most_profitable_segment_name}' segment:")
print("Top 5 Cities:")
print(df_profitable_customers['City'].value_counts().head())
print("\nTop 5 Countries:")
print(df_profitable_customers['Country'].value_counts().head())

# 3. What do they buy?
print(f"\n3. Products bought by customers in the '{most_profitable_segment_name}' segment:")
print("Top 5 Products:")
print(df_profitable_customers['Product'].value_counts().head())
print("\nTop 5 Product Categories (Category):")
print(df_profitable_customers['Category'].value_counts().head())
print("\nTop 5 Product Subcategories (Subcategory):")
print(df_profitable_customers['Subcategory'].value_counts().head())
```

IV. Conclusion

The "Promising" customer segment brings the highest profit to CosmeticWorld, with over 6,000 customers concentrated mainly in the United States, especially in major cities such as New York City, Los Angeles, and Philadelphia.

CosmeticWorld should prioritize marketing resources, customer care programs, and customer retention activities in these markets to maximize revenue and profit. At the same time, implementing loyalty programs and personalized offers will contribute to increasing repurchase rates and enhancing customer lifetime value.

Regarding shopping behavior, this customer group tends to spend a lot on personal care products, especially Body Care, Hair Care, and Make Up. CosmeticWorld should focus on developing these categories, and in future analyses, exploit more shopping cart data to identify products often purchased together to implement cross-selling programs and build suitable product bundles. In addition, it is necessary to analyze order value and purchase frequency by market to identify the most profitable areas, thereby optimizing marketing budgets and improving business efficiency.